KrishiMitra

AI-Powered Smart Farming Platform for Indian Agriculture

COMPLETE PROJECT DOCUMENTATION

Full-Stack Web Application

React · Express · PostgreSQL · FastAPI · TensorFlow

Version 1.0 | April 2026

Table of Contents

1 Introduction & Project Overview

1.1 What is KrishiMitra?

1.2 Complete Feature List

1.3 Technology Stack

2 System Architecture

2.1 High-Level Architecture Diagram

2.2 Folder Structure

3 Setup & Installation Guide

3.1 Prerequisites

3.2 Environment Variables

3.3 Database Setup

3.4 Backend Server Setup

3.5 Frontend Setup

3.6 ML Service Setup

4 Database Design

4.1 All Tables & Columns

4.2 Relationships

5 Backend (Express Server)

5.1 Server Configuration

5.2 Authentication System

5.3 Complete API Reference

5.4 Backend Services & Business Logic

6 Frontend (React App)

6.1 App Architecture & Routing

6.2 Design System

6.3 Page-by-Page Breakdown

6.4 Internationalization (i18n)

7 ML Service (Disease Detection)

8 External APIs Used

9 Data Flow Diagrams

10 Security & Best Practices

11 Deployment Guide

1. Introduction & Project Overview

1.1 What is KrishiMitra?

KrishiMitra (meaning "Farmer's Friend" in Hindi) is a full-stack AI-powered smart farming web application designed specifically for Indian farmers. It integrates real-time weather data, AI-based disease detection, government agricultural market data, and rule-based advisory systems to help farmers make better decisions about their crops.

The platform solves these key problems:

- **What crop should I grow?** — AI crop recommendation based on soil NPK, pH, weather
- **Is my crop sick?** — Upload a leaf photo and get instant AI disease diagnosis
- **When should I water?** — Smart irrigation scheduling with weather awareness
- **When to sell?** — Live market prices + 4-week price forecasts
- **Are pests coming?** — Weather-based pest and disease early warnings
- **How much will I harvest?** — Yield prediction based on farm conditions

> **Who This Documentation Is For**
>
> This documentation is written for anyone new to full-stack web development. Every concept, file, and decision is explained from the ground up. If you understand basic HTML, CSS, and JavaScript, you can follow along and implement this entire project.

1.2 Complete Feature List

#	FEATURE	WHAT IT DOES
1	User Authentication	Register/login with email & password. JWT-based session management.
2	Farm Management (CRUD)	Add, view, delete farms with details like crop, soil type, irrigation, GPS coordinates.
3	Crop & Fertilizer Advisory	Enter soil NPK + pH values → get crop recommendation and fertilizer guidance.
4	Crop Recommendation Engine	Score-match 18 crops against soil nutrients, weather, and pH. Shows top 8 matches.

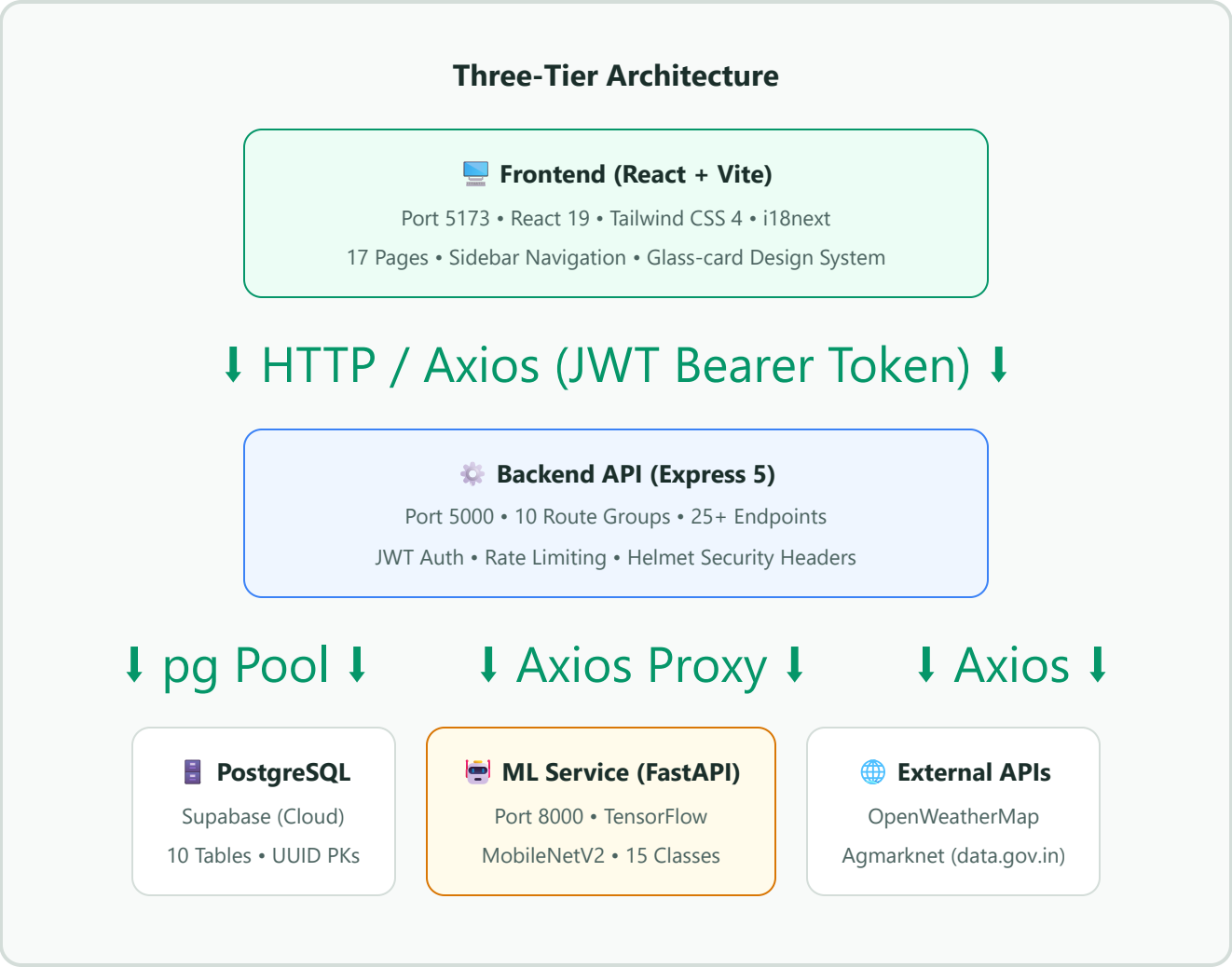
#	FEATURE	WHAT IT DOES
5	AI Disease Detection	Upload a leaf photo → TensorFlow MobileNetV2 model identifies 15 diseases across 3 crop types.
6	Disease Detection History	View all past disease analyses with confidence scores and treatments.
7	Smart Irrigation Scheduler	Generates a 28-day watering schedule based on crop, soil, growth stage, and live weather.
8	Pest & Disease Alerts	34 weather-based rules scan farm conditions for pest/disease risk. Severity-ranked alerts.
9	Yield Prediction	Estimates harvest quantity based on soil, irrigation, weather, and sowing season.
10	Live Market Prices	Search real-time commodity prices from India's Agmarknet (7,000+ mandis).
11	Price Forecast & Trends	4-week price prediction with sell/hold recommendation using moving averages.
12	Interactive Farm Map	Visualize all farms on a Leaflet.js map with popups showing farm details.
13	Reports & Export	Download farm data, advisories, disease reports, yield predictions as PDF/CSV.
14	Soil Test Lab Locator	Find nearest soil testing labs using GPS or state/district search. 35 real Indian labs.
15	Government Schemes	Dashboard widget showing active agricultural schemes with apply links.
16	Multi-language Support	Full UI translation in English, Hindi (हिन्दी), and Telugu (తెలుగు).
17	Responsive Design	Works on desktop, tablet, and mobile with sidebar/header layout.

1.3 Technology Stack

LAYER	TECHNOLOGY	VERSION	WHY THIS CHOICE
Frontend	React	19.2	Component-based UI library. Most popular for SPAs.
	Vite	7.3	Lightning-fast build tool. Replaces Create React App.
	Tailwind CSS	4.2	Utility-first CSS. Rapid styling without separate CSS files.
	React Router	7.13	Client-side routing for single-page application navigation.
	Axios	1.13	HTTP client with interceptors for auth token management.
	i18next	25.8	Internationalization framework for multi-language support.
Backend	Node.js	22.x	JavaScript runtime. Same language as frontend.
	Express	5.2	Minimal, flexible web framework for building REST APIs.
	PostgreSQL	—	Robust relational database. Supports JSONB for flexible data.
	JSON Web Tokens	9.0	Stateless authentication. No server sessions needed.
	bcryptjs	3.0	Password hashing. Never store passwords in plain text.
	Helmet	8.1	Security HTTP headers (XSS protection, content policy, etc.).
	Multer	2.0	File upload handling for disease detection images.
ML Service	Python / FastAPI	3.12	High-performance async API framework. Auto-generates docs.
	TensorFlow	2.x	Deep learning framework. Loads the Keras disease model.
	MobileNetV2	—	Lightweight CNN architecture. Good accuracy with small model size.
	Docker	—	Containerization for consistent deployment of the ML service.
Database Host	Supabase	—	Free PostgreSQL hosting with connection pooling.

2. System Architecture

2.1 High-Level Architecture Diagram



How It All Connects

The **React frontend** runs in the user's browser and communicates with the **Express backend** via REST API calls using Axios. The backend handles authentication, business logic, and database operations. For disease detection, the backend proxies image uploads to the **Python ML service**. For weather data and market prices, the backend calls **external APIs** (OpenWeatherMap and Agmarknet).

2.2 Folder Structure

```
KrishiMitra/
├── client/                        # React Frontend
│   ├── public/                    # Static files
│   ├── src/
│   │   ├── api/
│   │   │   └── axiosInstance.js     # Configured Axios with auth interceptor
│   │   ├── components/
│   │   │   ├── Navbar.jsx           # Sidebar + header navigation
│   │   │   ├── FarmForm.jsx         # Add farm form component
│   │   │   ├── FarmList.jsx         # Farm cards display component
│   │   │   └── GovernmentSchemes.jsx # Government schemes widget
│   │   ├── data/
│   │   │   └── locations.json       # Indian states/districts for dropdowns
│   │   ├── pages/
│   │   │   ├── Login.jsx            # Login page with hero panel
│   │   │   ├── Register.jsx         # Registration page
│   │   │   ├── Dashboard.jsx        # Main dashboard with feature cards
│   │   │   ├── Farms.jsx            # Farm CRUD management
│   │   │   ├── SimplifiedAdvisory.jsx # Crop & fertilizer advisory
│   │   │   ├── CropRecommendation.jsx # AI crop suggestions
│   │   │   ├── DiseaseDetection.jsx # Leaf image upload & AI analysis
│   │   │   ├── DiseaseHistory.jsx   # Past detection records
│   │   │   ├── IrrigationScheduler.jsx # Smart watering schedule
│   │   │   ├── PestAlerts.jsx       # Weather-based pest warnings
│   │   │   ├── YieldPrediction.jsx  # Harvest yield estimates
│   │   │   ├── MarketPrices.jsx     # Live Agmarknet prices
│   │   │   ├── PriceForecast.jsx    # 4-week price trends
│   │   │   ├── FarmMap.jsx          # Interactive Leaflet map
│   │   │   ├── ReportExport.jsx     # PDF/CSV data export
│   │   │   ├── SoilLabLocator.jsx   # Find nearby soil labs
│   │   │   └── Roadmap.jsx          # Platform roadmap & vision
│   │   ├── App.jsx                  # Root component with routing
│   │   ├── App.css                  # (empty — styles are in index.css)
│   │   ├── index.css                # Complete design system
│   │   ├── i18n.js                  # Translation config (EN/HI/TE)
│   │   └── main.jsx                 # React entry point
│   ├── index.html                   # HTML shell
│   ├── package.json                 # Frontend dependencies
│   ├── vite.config.js               # Vite configuration
│   ├── tailwind.config.js           # Tailwind configuration
│   └── postcss.config.js            # PostCSS configuration
│
├── server/                        # Express Backend
│   ├── config/
│   │   └── db.js                    # PostgreSQL pool configuration
```

```
│   ├── controllers/
│   │   ├── authController.js        # Register & login logic
│   │   ├── farmController.js        # Farm CRUD operations
│   │   └── advisoryController.js    # Simplified advisory + weather endpoint
│   ├── data/
│   │   └── advisoryRules.json       # Rule-based advisory conditions
│   ├── middleware/
│   │   └── authMiddleware.js        # JWT verification middleware
│   ├── models/
│   │   └── db.sql                   # Database schema (CREATE TABLE statements)
│   ├── routes/
│   │   ├── authRoutes.js            # /api/auth/*
│   │   ├── farmRoutes.js            # /api/farms/*
│   │   ├── advisoryRoutes.js        # /api/advisories/*
│   │   ├── marketRoutes.js          # /api/market/*
│   │   └── mlRoutes.js              # /api/disease/*
│   ├── services/
│   │   ├── advisoryEngine.js        # Rule evaluation engine
│   │   ├── weatherService.js        # OpenWeatherMap integration
│   │   ├── cropRecommendation.js    # Crop scoring algorithm
│   │   ├── irrigationScheduler.js   # 28-day schedule generator
│   │   ├── pestAlertService.js      # 34-rule pest warning system
│   │   ├── yieldPredictor.js        # Yield estimation engine
│   │   ├── priceForecaster.js       # Moving average forecaster
│   │   └── soilLabLocator.js        # Lab search with GPS/Haversine
│   ├── index.js                     # Server entry point
│   ├── package.json                 # Backend dependencies
│   └── .env                         # Environment variables (not in git)
│
└── ml_service/                      # Python ML Service
    ├── model/
    │   ├── plant_disease_model.h5   # Trained TensorFlow model (~25MB)
    │   └── class_names.json         # 15 disease class labels
    ├── main.py                      # FastAPI server with /predict endpoint
    ├── requirements.txt             # Python dependencies
    └── Dockerfile                   # Container configuration
```

3. Setup & Installation Guide

This section walks you through setting up the entire project from scratch on your local machine.

3.1 Prerequisites

Install these tools before starting:

TOOL	MINIMUM VERSION	DOWNLOAD FROM	WHAT IT'S FOR
Node.js	20.19+	nodejs.org	Runs the backend server and frontend build tool
npm	10+	(comes with Node.js)	Package manager for JavaScript dependencies
Python	3.10+	python.org	Runs the ML disease detection service
pip	—	(comes with Python)	Package manager for Python dependencies
Git	2.x	git-scm.com	Version control and cloning the repository
PostgreSQL	14+	Supabase.com (free)	The database. You can use a cloud-hosted instance.

3.2 Environment Variables

Create a file named `.env` inside the `server/` folder with these variables:

```
# server/.env

# Server port (optional, defaults to 5000)
PORT=5000

# PostgreSQL connection string (REQUIRED)
# Get this from Supabase → Project Settings → Database → Connection String
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@db.YOUR_PROJECT.supabase.co:5432/postgres

# Secret key for signing JWT tokens (REQUIRED for security)
# Use any long random string. Example: openssl rand -hex 32
JWT_SECRET=your_super_secret_random_string_here

# OpenWeatherMap API key (REQUIRED for weather features)
# Sign up free at openweathermap.org/api
OPENWEATHER_API_KEY=your_openweather_api_key

# ML Service URL (optional, defaults to http://127.0.0.1:8000)
ML_SERVICE_URL=http://127.0.0.1:8000

# Agmarknet API key (optional, has a fallback)
# Get from data.gov.in
AGMARKNET_API_KEY=your_agmarknet_api_key
```

Important: Never Commit .env Files

The `.env` file contains sensitive credentials (database password, API keys). It should be listed in `.gitignore` and never pushed to GitHub.

3.3 Database Setup

Option A: Using Supabase (Recommended — Free)

1. Go to supabase.com and create a free account
2. Click "New Project" → give it a name and password
3. Once created, go to **SQL Editor**
4. Copy the entire contents of `server/models/db.sql` and paste it into the SQL editor
5. Click "Run" — this creates all the base tables (users, farms, advisories, disease_reports, market_prices)

6. Then run the Phase 1 migration SQL to create the additional tables (crop_recommendations, irrigation_schedules, pest_alerts, yield_predictions, etc.)
7. Go to **Settings** → **Database** → **Connection String** and copy the URI. Put it in your `.env` as `DATABASE_URL`

Option B: Local PostgreSQL

1. Install PostgreSQL from postgresql.org
2. Open pgAdmin or psql terminal
3. Create a database: `CREATE DATABASE krishimitra;`
4. Run the SQL files listed above
5. Set `DATABASE_URL=postgresql://postgres:password@localhost:5432/krishimitra`

3.4 Backend Server Setup

```
# 1. Navigate to the server folder
cd KrishiMitra/server

# 2. Install all Node.js dependencies
npm install

# 3. Make sure your .env file is in place (see section 3.2)

# 4. Start the development server (with auto-restart on file changes)
npm run dev

# You should see:
# Server running on port 5000
# DB connected: [timestamp]
```

> **What `npm install` Does**
>
> It reads `package.json`, downloads all listed dependencies (Express, pg, bcryptjs, etc.) into a `node_modules/` folder. You only need to run this once (or after adding new dependencies).

3.5 Frontend Setup

```
# 1. Navigate to the client folder
cd KrishiMitra/client

# 2. Install dependencies
npm install

# 3. Start the Vite development server
npm run dev

# You should see:
# VITE v7.3.1  ready in 300ms
# ➜ Local: http://localhost:5173/
```

The frontend connects to the backend at `http://localhost:5000` by default (configured in `axiosInstance.js`). If your backend runs on a different port, create a `client/.env` file:

```
# client/.env (optional)
VITE_API_URL=http://localhost:5000
```

3.6 ML Service Setup

```
# 1. Navigate to the ML service folder
cd KrishiMitra/ml_service

# 2. Create a Python virtual environment (recommended)
python -m venv venv

# 3. Activate the virtual environment
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# 4. Install Python dependencies
pip install -r requirements.txt

# 5. Start the FastAPI server
python main.py

# You should see:
# Loading Custom Keras Model...
# Model loaded successfully.
# Loaded 15 classes.
# Uvicorn running on http://0.0.0.0:8000
```

TensorFlow Installation Note

TensorFlow is a large package (~500MB). The first install may take several minutes. If you have a GPU, install `tensorflow-gpu` for faster inference. On Apple Silicon Macs, use `tensorflow-macos`.

4. Database Design

4.1 All Tables & Columns

users

Stores registered user accounts.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
`id`	UUID	PRIMARY KEY, auto-generated	Unique user identifier
`name`	VARCHAR(255)	NOT NULL	User's display name
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Login email (must be unique)
`password_hash`	VARCHAR(255)	NOT NULL	bcrypt-hashed password (never plain text)
`role`	VARCHAR(50)	DEFAULT 'farmer'	Either 'farmer' or 'admin'
`created_at`	TIMESTAMPTZ	DEFAULT now()	Registration timestamp

farms

Stores farm registrations with crop, soil, location, and irrigation details.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
`id`	UUID	PRIMARY KEY	Unique farm identifier
`user_id`	UUID	FK → users(id) CASCADE	Owner of this farm
`farm_name`	VARCHAR(255)	NOT NULL	Name of the farm
`area_acres`	NUMERIC	CHECK > 0	Farm area in acres
`soil_type`	VARCHAR(100)	NOT NULL	Alluvial, Black, Red, Sandy, Loamy, Clay, Laterite
`current_crop`	VARCHAR(100)	NOT NULL	Currently planted crop

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
`soil_ph`	NUMERIC	CHECK 3.0–9.0	Soil pH level
`irrigation_type`	VARCHAR(100)	nullable	Drip, Sprinkler, Flood, Canal, Rainfed
`sowing_date`	DATE	nullable	When the crop was sown
`previous_crop`	VARCHAR(100)	nullable	Last season's crop (for rotation analysis)
`latitude`	NUMERIC	nullable	GPS latitude (for map & weather)
`longitude`	NUMERIC	nullable	GPS longitude
`district`	VARCHAR(100)	nullable	Indian district name
`state`	VARCHAR(100)	nullable	Indian state name
`created_at`	TIMESTAMPTZ	DEFAULT now()	When the farm was registered

disease_reports

Stores AI disease detection results from leaf image analysis.

COLUMN	TYPE	DESCRIPTION
`id`	UUID	Primary key
`user_id`	UUID (FK)	Which user ran this detection
`predicted_disease`	VARCHAR(255)	The AI's predicted disease name
`confidence`	NUMERIC	Confidence percentage (0-100)
`treatment`	TEXT	Recommended treatment text
`low_confidence`	BOOLEAN	True if confidence was below 60%
`created_at`	TIMESTAMPTZ	When the analysis was performed

Other Tables (Phase 1)

TABLE	KEY COLUMNS	PURPOSE
`crop_recommendations`	user_id, farm_id, N/P/K/temp/humidity/pH/rainfall, recommended_crops (JSONB)	Stores crop recommendation history

TABLE	KEY COLUMNS	PURPOSE
`irrigation_schedules`	user_id, farm_id, crop, schedule (JSONB), next_irrigation	Generated irrigation schedules
`pest_alerts`	farm_id, user_id, alert_type, severity, pest_disease, message	Weather-based pest/disease alerts
`yield_predictions`	user_id, farm_id, crop, predicted_yield_kg, harvest_window_start/end	Yield estimates with harvest timing
`simplified_advisories`	user_id, N/P/K/temp/humidity/pH/rainfall, crop_recommendation, fertilizer_advice	Simple advisory results
`market_price_history`	commodity, state, district, modal/min/max_price, recorded_at	Historical price data for forecasting
`advisories`	farm_id, weather_snapshot (JSONB), rules_triggered (JSONB), advisories_given (JSONB)	Detailed rule-based advisory results
`market_prices`	commodity, state, district, modal/min/max_price, fetched_at	Cached market price data

4.2 Relationships

All relationships use UUID foreign keys with `ON DELETE CASCADE` (if a user is deleted, all their farms, advisories, etc. are automatically removed).

- **users** → has many → farms, disease_reports, crop_recommendations, irrigation_schedules, pest_alerts, yield_predictions, simplified_advisories
- **farms** → has many → advisories, crop_recommendations, irrigation_schedules, pest_alerts, yield_predictions

5. Backend (Express Server)

5.1 Server Configuration

The server entry point (`server/index.js`) sets up:

FEATURE	LIBRARY	CONFIGURATION
CORS	`cors`	Allow all origins (for development). In production, restrict to your frontend domain.
Security Headers	`helmet`	Adds XSS protection, content-type sniffing prevention, etc.
Body Parser	`express.json()`	Parses incoming JSON request bodies.
Rate Limiting	`express-rate-limit`	100 requests per IP per 15 minutes on `/api/*` routes.
Health Check	—	`GET /health` → `{ status: 'ok' }`

Middleware Execution Order

Request In → CORS → Helmet → JSON Parser → Rate Limiter → Route Handler

5.2 Authentication System

How JWT Authentication Works

JWT (JSON Web Token) is a stateless authentication method. Instead of storing sessions on the server, we give the user a signed token that proves their identity.

1. User sends email + password → 2. Server verifies password →
3. Server creates JWT with user ID → 4. Token sent back to browser

5. Browser stores token in localStorage → 6. Every API call includes token in header →

7. Server verifies token → knows who's asking

Registration Flow

```
POST /api/auth/register
Body: { "name": "Ravi Kumar", "email": "ravi@example.com", "password": "secret123" }

Backend does:
1. Validates: name not empty, email format valid, password ≥ 6 chars
2. Checks if email already exists in database
3. Hashes password with bcrypt (10 salt rounds)
4. Inserts new user into 'users' table
5. Creates JWT token (expires in 7 days)
6. Returns: { token: "eyJ...", user: { id, name, email, role } }
```

Login Flow

```
POST /api/auth/login
Body: { "email": "ravi@example.com", "password": "secret123" }

Backend does:
1. Finds user by email in database
2. Compares password with stored hash using bcrypt
3. If match → creates JWT token
4. Returns: { token: "eyJ...", user: { id, name, email, role } }
```

Auth Middleware (Protecting Routes)

```
// Every protected route passes through this middleware:
function authMiddleware(req, res, next) {
    1. Read 'Authorization' header from request
    2. Strip 'Bearer ' prefix → extract token
    3. Verify token signature using JWT_SECRET
    4. Extract user ID from token payload
    5. Set req.user = { id, role }
    6. Call next() → allow the route handler to execute

    // If token is missing or invalid → return 401 Unauthorized
}
```

5.3 Complete API Reference

Auth Endpoints (Public — No Token Required)

METHOD	ENDPOINT	REQUEST BODY	RESPONSE
POST	`/api/auth/register`	`{ name, email, password }`	`{ token, user: { id, name, email, role } }`
POST	`/api/auth/login`	`{ email, password }`	`{ token, user: { id, name, email, role } }`

Farm Endpoints (Private — Token Required)

METHOD	ENDPOINT	REQUEST	RESPONSE
POST	`/api/farms`	Body: `{ farm_name, area_acres, soil_type, current_crop, soil_ph?, irrigation_type?, sowing_date?, latitude?, longitude?, district?, state? }`	Created farm object
GET	`/api/farms`	—	Array of user's farms
GET	`/api/farms/:id`	Farm UUID in URL	Single farm object
DELETE	`/api/farms/:id`	Farm UUID in URL	`{ msg: "Farm removed" }`

Advisory Endpoints (Private)

METHOD	ENDPOINT	REQUEST	RESPONSE
POST	`/api/advisories/simplified`	Body: `{ nitrogen, phosphorous, potassium, temperature, humidity, ph, rainfall }`	`{ crop_recommendation, fertilizer_advice }`
GET	`/api/advisories/history`	—	Array of past advisories
GET	`/api/advisories/weather?city=X`	City name as query param	`{ temperature, humidity, rainfall }`

Disease Detection Endpoints (Private)

METHOD	ENDPOINT	REQUEST	RESPONSE
POST	`/api/disease/detect`	multipart/form-data with `image` field (max 5MB, image/* only)	`{ disease, confidence, treatment, low_confidence }`
GET	`/api/disease/history`	—	Last 30 detection records

Market Endpoints (Private)

METHOD	ENDPOINT	REQUEST	RESPONSE
GET	`/api/market/prices?state=X&district=Y&commodity=Z`	All 3 query params	`{ bestMarket, otherMarkets[], fallbackTriggered }`
GET	`/api/market/forecast?commodity=X&state=Y`	Both query params	`{ currentPrice, trend, forecast[], recommendation }`

Crop Recommendation Endpoints (Private)

METHOD	ENDPOINT	REQUEST	RESPONSE
POST	`/api/crop-recommend`	Body: `{ nitrogen, phosphorous, potassium, temperature, humidity, ph, rainfall, soil_type? }`	`{ recommendations: [top 8 crops] }`
POST	`/api/crop-recommend/from-farm/:farmId`	Farm UUID in URL	`{ recommendations, weather, farm }`

Other Endpoints (Private)

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/irrigation/schedule/:farmId`	Generate 28-day irrigation schedule for a farm

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/pest-alerts/all`	Scan ALL user farms for pest/disease risks
GET	`/api/pest-alerts/farm/:farmId`	Scan specific farm for alerts
POST	`/api/yield/predict/:farmId`	Predict yield and harvest window for a farm
GET	`/api/soil-labs/search?latitude=X&longitude=Y`	Find soil labs by GPS coordinates
GET	`/api/soil-labs/search?state=X&district=Y`	Find soil labs by state/district

5.4 Backend Services & Business Logic

Crop Recommendation Engine

Located in `services/cropRecommendation.js`. Uses a scoring algorithm to match user inputs against 18 Indian crops.

```
Algorithm:
1. For each of the 18 crops (Rice, Wheat, Maize, Cotton, etc.):
   a. Score Nitrogen: How close is user's N to crop's ideal N range? (0-1 scale)
   b. Score Phosphorous: same approach (0-1)
   c. Score Potassium: same (0-1)
   d. Score Temperature: same (0-1)
   e. Score Humidity: same (0-1)
   f. Score pH: same (0-1)
   g. Score Rainfall: same (0-1)
   h. Bonus if soil_type matches crop's preferred soils (+1 point)

2. matchPercent = (totalScore / maxScore) × 100

3. Sort all crops by matchPercent DESC

4. Return top 8 crops with: crop name, matchPercent, compatible seasons,
   ideal conditions, soil types, expected yield per acre
```

Irrigation Scheduling Engine

Located in `services/irrigationScheduler.js`. Generates a 28-day watering plan.

```
Algorithm:
1. Look up crop's water needs (mm/day) for current growth stage
   - Growth stage is calculated from sowing_date:
     <30 days = seedling, <60 = vegetative, <90 = flowering, ≥90 = maturity

2. Adjust for soil retention factor:
   - Clay holds water well (factor 1.3) → needs less frequent watering
   - Sandy soil drains fast (factor 0.6) → needs more frequent

3. Adjust for irrigation efficiency:
   - Drip (90% efficient) → least water waste
   - Flood (50% efficient) → needs much more water

4. Adjust for live weather:
   - Temperature > 35°C → +30% water need
   - Humidity > 80% → -15% water need
   - Rain forecast > 20mm → -50% water need (skip irrigation)

5. Calculate interval (every 2-7 days based on effective need)

6. Generate 28 daily entries: date, water amount, "scheduled" or "skip"
```

Pest Alert Service

Located in `services/pestAlertService.js`. Contains 34 weather-based pest/disease rules.

```
Rule Examples:
- Rice + Humidity > 85% + Temp 25-35°C → "Rice Blast" (HIGH severity)
- Cotton + Rainfall > 30mm            → "Bollworm Risk" (CRITICAL)
- Tomato + Humidity > 75% + Temp < 20°C → "Late Blight" (HIGH)
- Any Crop + Temp > 40°C              → "Heat Stress" (CRITICAL)
- Any Crop + Rainfall > 50mm          → "Waterlogging" (HIGH)

Each rule has: condition function, severity level, pest/disease name,
               advisory message, and applicable crop(s)
```

Yield Predictor

Located in `services/yieldPredictor.js`. Estimates harvest quantity using multiplicative factors.

```
Algorithm:
Start with yieldFactor = 1.0, then multiply by:

1. Soil type factor: e.g., Rice on Alluvial soil = 1.1, on Sandy = 0.7
2. pH factor: ideal 5.5-7.5 = 1.0, extreme values = 0.7
3. Irrigation factor: Drip = 1.1, Sprinkler = 1.0, Rainfed = 0.65
4. Weather factor: Extreme heat = 0.7, Moderate = 0.85
5. Season factor: Right sowing season = 1.0, Wrong = 0.75

Final: predicted_yield = optimal_yield × yieldFactor × farm_area
Confidence: HIGH if yieldFactor > 0.85, else MEDIUM
```

Price Forecaster

Located in `services/priceForecaster.js`. Uses moving averages for 4-week price predictions.

```
Algorithm:
1. Load historical prices from database (last 60 records)
   - If < 5 records, use seasonal baseline prices (15 commodities built-in)

2. Calculate Short-term Moving Average (3-point) = recent trend
3. Calculate Long-term Moving Average (7-point) = overall trend

4. Determine trend:
   - Short MA > Long MA × 1.03 → "rising"
   - Short MA < Long MA × 0.97 → "falling"
   - Otherwise → "stable"

5. Forecast 4 weeks: extrapolate from recent change, dampened over time

6. Sell/Hold recommendation:
   - Falling + current > forecast avg → "Sell now"
   - Rising → "Hold for better prices"
   - Stable → compare to short-term average
```

6. Frontend (React App)

6.1 App Architecture & Routing

How the React App Is Organized

React uses a component-based architecture. The entire UI is built from reusable components — small building blocks (like a button or a card) that are composed into larger components (like a page).

main.jsx → App.jsx → AppLayout → Navbar + Page

Routing Table

`App.jsx` uses React Router to map URL paths to page components:

URL PATH	COMPONENT	NAVBAR?	DESCRIPTION
`/`	Login	No	Login page (default landing)
`/login`	Login	No	Login page
`/register`	Register	No	Create account page
`/dashboard`	Dashboard	Yes	Main overview with feature cards and tips
`/farms`	Farms	Yes	Add/view/delete farms
`/advisory`	SimplifiedAdvisory	Yes	Soil NPK/pH crop recommendation
`/crop-recommend`	CropRecommendation	Yes	AI crop matching engine
`/disease-detection`	DiseaseDetection	Yes	Upload leaf photo for AI analysis
`/disease-history`	DiseaseHistory	Yes	Past detection records
`/irrigation`	IrrigationScheduler	Yes	Smart watering schedule
`/pest-alerts`	PestAlerts	Yes	Weather-based pest warnings
`/yield-prediction`	YieldPrediction	Yes	Harvest estimates
`/market-prices`	MarketPrices	Yes	Live Agmarknet commodity prices
`/price-forecast`	PriceForecast	Yes	Price trend analysis and 4-week forecast

URL PATH	COMPONENT	NAVBAR?	DESCRIPTION
`/farm-map`	FarmMap	Yes	Interactive map of all farms
`/reports`	ReportExport	Yes	Export data as PDF/CSV
`/soil-labs`	SoilLabLocator	Yes	Find nearby soil testing labs
`/roadmap`	Roadmap	Yes	Platform future vision

Auth Protection

There is no `<PrivateRoute>` wrapper component. Instead, authentication is enforced via the **Axios response interceptor**: if any API call returns a `401 Unauthorized` response, the interceptor automatically clears localStorage and redirects to `/login`. Individual pages also check for a token on mount and redirect if missing.

Axios Instance Configuration

```
// api/axiosInstance.js
const api = axios.create({
    baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000'
});

// Request interceptor: attach JWT to every request
api.interceptors.request.use(config => {
    const token = localStorage.getItem('token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});

// Response interceptor: handle 401
api.interceptors.response.use(
    response => response,
    error => {
        if (error.response?.status === 401) {
            localStorage.removeItem('token');
            localStorage.removeItem('user');
            window.location.href = '/login';
        }
        return Promise.reject(error);
    }
);
```

6.2 Design System

All styles live in `src/index.css`. The design system uses CSS custom properties (variables) and reusable utility classes.

Color Palette

VARIABLE	VALUE	USAGE
`--color-primary`	#065f46	Main brand green (headings, accents)
`--color-primary-light`	#059669	Interactive elements, links
`--color-accent`	#d97706	Amber accent for highlights
`--color-bg`	#f0f4f3	Page background
`--color-text`	#1a2e2a	Primary text color
`--color-border`	#d4ddd9	Borders and dividers

Key CSS Classes

CLASS	WHAT IT DOES	WHERE USED
`.glass-card`	White card with 85% opacity, blur backdrop, subtle shadow. Hover elevates.	Every card/panel on every page
`.gradient-primary`	Green gradient background (3-stop emerald)	Logo icon, buttons, header elements
`.gradient-hero`	Darker green gradient for full panels	Login/Register hero section
`.btn-primary`	Green gradient button with hover lift effect	Primary action buttons everywhere
`.btn-secondary`	White outlined button with green text	Secondary actions
`.input-field`	Full-width input with green focus ring	All form inputs
`.page-title`	Large bold heading (1.625rem, weight 800)	Top of every page
`.page-subtitle`	Smaller muted description text	Below page titles
`.page-content`	Max 1280px centered container with padding	Wraps every page's content
`.badge-success`	Green pill badge	Status indicators

CLASS	WHAT IT DOES	WHERE USED
`.skeleton`	Shimmer loading animation placeholder	Loading states

Layout Architecture

```
┌──────────────────────────────────────────────────┐
│  Header Bar (64px height, sticky top)            │
│  [☰ Menu] [🌿 KrishiMitra] ........... [EN▾] [👤] [Logout] │
├──────────┬───────────────────────────────────────┤
│ Sidebar  │  Page Content                         │
│ (260px)  │  ┌──────────────────────────────┐     │
│          │  │ .page-content (max 1280px)   │     │
│ Overview │  │                              │     │
│ • Dash   │  │  page-title                  │     │
│ • Farms  │  │  page-subtitle               │     │
│          │  │                              │     │
│ Crop Int │  │  ┌──────┐ ┌──────┐ ┌──────┐  │     │
│ • Advis  │  │  │glass │ │glass │ │glass │  │     │
│ • CropR  │  │  │-card │ │-card │ │-card │  │     │
│ • Diseas │  │  └──────┘ └──────┘ └──────┘  │     │
│ • Pest   │  │                              │     │
│          │  └──────────────────────────────┘     │
│ Farm Mgmt│                                       │
│ • Irrig  │  On mobile: sidebar becomes overlay   │
│ • Yield  │  triggered by hamburger menu          │
│ • Map    │                                       │
│          │                                       │
│ Market   │                                       │
│ • Prices │                                       │
│ • Foreca │                                       │
│          │                                       │
│ Tools    │                                       │
│ • Labs   │                                       │
│ • Histor │                                       │
│ • Report │                                       │
│ • Roadma │                                       │
│          │                                       │
│ ──────── │                                       │
│ Need Help│                                       │
│ tip box  │                                       │
└──────────┴───────────────────────────────────────┘
```

6.3 Page-by-Page Breakdown

Login & Register Pages

Split-screen layout: left 55% is a gradient hero panel with branding and feature highlights; right 45% is the form. On mobile, the hero panel hides and only the form is shown.

Dashboard

The main landing page after login. Shows a greeting (Good morning/afternoon/evening + user name), a 3-column grid of 11 feature cards (each links to its respective page), a Government Schemes widget, and a Seasonal Farming Tips section.

Farms (CRUD)

Expandable "Add New Farm" form at the top. Below it, a grid of farm cards showing name, crop, area, soil, pH, irrigation, sowing date, and coordinates. Each card has a delete button. Form fields: farm name, area (acres), soil type (dropdown), current crop (dropdown), soil pH, irrigation type, sowing date, previous crop, latitude/longitude, district, state.

Crop Advisory (SimplifiedAdvisory)

Two sections: (1) Input form with NPK sliders/inputs, pH, and weather data (temperature, humidity, rainfall). Has an "Auto-fill" button that fetches live weather. (2) Results panel showing a Soil Health Score (Good/Fair/Poor based on NPK ranges), the recommended crop, and fertilizer advice. Includes tabs for "Latest Result" and "History".

Crop Recommendation

Two tabs: "Manual Input" (enter N/P/K/temp/humidity/pH/rainfall manually) and "From My Farm" (select a registered farm to auto-populate). Results show top 8 matching crops with match percentage, compatible seasons, and expected yield.

Disease Detection

Drag-and-drop image upload area for leaf photos. Supports JPG, PNG, WEBP up to 5MB. After analysis, shows detected disease name, confidence score (with color-coded badge), and recommended treatment. Below the main area is a "How Disease Detection Works" section explaining the 3-step process. Links to Disease History page.

Irrigation Scheduler

Farm dropdown selector. Generates a 28-day schedule showing: Schedule Summary (growth stage, water need/day, interval, estimated harvest date), Weather Advisory with current conditions, and a day-by-day schedule table with water amounts and skip/scheduled status.

Pest Alerts

Farm multi-select (or "All My Farms"). Shows severity-color-coded alert cards with pest/disease name, type (Fungal/Bacterial/Viral/Pest/Abiotic), advisory message, and weather snapshot used for the evaluation.

Yield Prediction

Farm dropdown. Shows predicted yield in kg, yield per acre, percentage of optimal, confidence level, impact factors breakdown (soil, pH, irrigation, weather, season), harvest window dates, and a visual comparison bar (Minimum ↔ Your Prediction ↔ Optimal).

Market Prices

Search interface with state, district, and commodity dropdowns. Shows results in a card grid with min/max/modal prices per quintal. Summary stats at top (best price, price range, markets found). Agmarknet data source with fallback notice.

Price Forecast

Commodity + State selectors. Shows current price, trend direction (rising/falling/stable), short & long-term averages, a trend chart (historical + 4-week forecast), Sell/Hold recommendation with explanation, and weekly forecast breakdown.

Farm Map

Interactive Leaflet.js map showing all farms with lat/lon as markers. Sidebar listing farms. Click a farm to see popup with details. Farms without coordinates are listed separately with a reminder to add them.

Reports & Export

Four export sections: Farm Details, Crop Advisories, Disease Detections, Yield Predictions. Each has CSV and PDF download buttons. Plus a "Complete Farm Report" button that generates a comprehensive PDF with all data.

Soil Lab Locator

Two search modes: GPS-based (uses device location + Haversine distance) and State/District selection. Results show lab name, type (Govt/Private), distance, address, phone, fee, turnaround time, and services. Includes a "How to Collect Sample" guide and Google Maps links.

6.4 Internationalization (i18n)

The entire UI supports three languages, switchable via a dropdown in the header:

CODE	LANGUAGE	SCRIPT
`en`	English	Latin
`hi`	Hindi	Devanagari (हिन्दी)
`te`	Telugu	Telugu script (తెలుగు)

How it works: All user-facing strings in JSX files are wrapped in `t('key')` calls from react-i18next. The translation strings for all three languages are defined inline in `src/i18n.js` (~1290 lines, ~350 keys per language). The Roadmap page is intentionally excluded from translation.

```
// Example: In a JSX component
import { useTranslation } from 'react-i18next';

function MyComponent() {
    const { t } = useTranslation();
    return <h1>{t('Dashboard')}</h1>;
    // Renders "Dashboard" in English
    // Renders "डैशबोर्ड" in Hindi
    // Renders "డాష్‌బోర్డ్" in Telugu
}
```

7. ML Service (Disease Detection)

Overview

A standalone Python microservice built with FastAPI that performs plant disease classification using a pre-trained TensorFlow deep learning model.

Model Details

PROPERTY	VALUE
Architecture	MobileNetV2 (Convolutional Neural Network)
Input Size	224 × 224 pixels, RGB
Output	15 class probabilities (softmax)
Model Format	Keras H5 (.h5 file)
Model File Size	~25 MB
Confidence threshold	60% (below = "Uncertain" result)

Supported Diseases (15 Classes)

#	CLASS NAME	CROP	DISEASE
1	Pepper__bell___Bacterial_spot	Bell Pepper	Bacterial Spot
2	Pepper__bell___healthy	Bell Pepper	Healthy
3	Potato___Early_blight	Potato	Early Blight
4	Potato___Late_blight	Potato	Late Blight
5	Potato___healthy	Potato	Healthy
6	Tomato_Bacterial_spot	Tomato	Bacterial Spot
7	Tomato_Early_blight	Tomato	Early Blight
8	Tomato_Late_blight	Tomato	Late Blight

#	CLASS NAME	CROP	DISEASE
9	Tomato_Leaf_Mold	Tomato	Leaf Mold
10	Tomato_Septoria_leaf_spot	Tomato	Septoria Leaf Spot
11	Tomato_Spider_mites	Tomato	Spider Mites
12	Tomato__Target_Spot	Tomato	Target Spot
13	Tomato__YellowLeaf_Curl_Virus	Tomato	Yellow Leaf Curl Virus
14	Tomato__Tomato_mosaic_virus	Tomato	Mosaic Virus
15	Tomato_healthy	Tomato	Healthy

API Endpoints

METHOD	PATH	DESCRIPTION
GET	`/health`	Returns model and class loading status
POST	`/predict`	Accepts image file, returns disease prediction

Prediction Pipeline

1. Receive Image → 2. Convert to RGB → 3. Resize to 224×224 → 4. Normalize [0,1] → 5. Model Predict → 6. Get Top Class

```
def preprocess_image(image_bytes):
    img = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    img = img.resize((224, 224))
    img_array = np.array(img) / 255.0   # Normalize pixel values to 0-1
    img_array = np.expand_dims(img_array, axis=0)  # Add batch dimension
    return img_array  # Shape: (1, 224, 224, 3)

# Prediction
predictions = model.predict(processed_img)     # Shape: (1, 15) — probabilities
predicted_idx = np.argmax(predictions[0])       # Index of highest probability
confidence = predictions[0][predicted_idx] * 100  # Convert to percentage

# If confidence < 60% → return "Uncertain" with advice to retake photo
# If confidence ≥ 60% → return disease name + treatment recommendation
```

Treatment Mapping

Each of the 15 classes has a hardcoded treatment recommendation. For example:

- **Tomato Late Blight**: "Apply specific fungicides immediately. Destroy heavily infected plants to stop spread."
- **Potato Early Blight**: "Apply fungicides like chlorothalonil or mancozeb. Practice crop rotation."
- **Tomato Spider Mites**: "Spray with insecticidal soap or neem oil. Introduce predatory mites."

Docker Deployment

```
# Build the Docker image
docker build -t krishimitra-ml .

# Run the container
docker run -p 8000:8000 krishimitra-ml

# The service will be available at http://localhost:8000
```

8. External APIs Used

API	PROVIDER	USED FOR	HOW TO GET KEY	FREE TIER
OpenWeatherMap	openweathermap.org	Current weather (temp, humidity, rainfall) for farm locations. Used by irrigation, pest alerts, yield, and crop recommendation.	Sign up → API Keys section → Copy key	60 calls/minute, 1M calls/month
Agmarknet	data.gov.in	Real-time agricultural commodity prices across 7,000+ Indian mandis (markets).	Register at data.gov.in → Request API key for the commodity prices dataset	Generous free tier
OpenWeatherMap Geocoding	openweathermap.org	Convert district/state names to latitude/longitude when farm doesn't have GPS coordinates.	Same key as above	Included in free tier

9. Data Flow Diagrams

Disease Detection Flow

User uploads leaf image → React sends to /api/disease/detect → Express checks JWT → Multer processes upload (max 5MB) → Express health-checks ML service → Express proxies image to FastAPI /predict → TensorFlow model inference → Result + treatment returned → Saved to disease_reports table → Displayed in UI

Crop Recommendation Flow

User enters NPK + pH → OR selects a farm → Backend fetches live weather → 18 crops scored on 7 parameters → Top 8 returned as % match → Saved + displayed

Market Prices Flow

User selects commodity+state+district → Backend calls Agmarknet API → Try district filter first → Fallback to state-wide if empty → Parse + rank results → Show best market + others

10. Security & Best Practices

MEASURE	IMPLEMENTATION	WHY IT MATTERS
Password Hashing	bcrypt with 10 salt rounds	Even if database is leaked, passwords cannot be reversed.
JWT Authentication	Signed tokens with secret key, 7-day expiry	Stateless auth. No server-side sessions to manage.
Helmet Security Headers	XSS, CSP, content-type sniffing protection	Prevents many common web attacks automatically.
Rate Limiting	100 requests per IP per 15 minutes	Prevents brute-force attacks and API abuse.
Input Validation	express-validator on auth routes	Prevents SQL injection and malformed data.
SQL Parameterized Queries	All DB queries use `$1, $2` placeholders	Prevents SQL injection attacks.
File Upload Limits	Multer: 5MB max, image MIME types only	Prevents malicious file uploads.
User Scoping	All queries filter by `user_id`	Users can only see their own data.
CORS	Currently open (*) for development	Should be restricted to frontend domain in production.
Environment Variables	Secrets in .env file, not in code	Credentials never committed to version control.

11. Deployment Guide

Recommended Deployment Architecture

COMPONENT	RECOMMENDED HOST	FREE TIER?
Frontend (React)	Vercel, Netlify, or Cloudflare Pages	Yes
Backend (Express)	Render, Railway, or Fly.io	Yes (with limits)

COMPONENT	RECOMMENDED HOST	FREE TIER?
Database (PostgreSQL)	Supabase or Neon	Yes (500MB–1GB)
ML Service (FastAPI)	Render (Docker), Railway, or Fly.io	Yes (with limits)

Frontend Deployment (Vercel)

```
# 1. Push your code to GitHub

# 2. Go to vercel.com → Import Project → Select your repo

# 3. Set these settings:
   Root Directory: KrishiMitra/client
   Build Command: npm run build
   Output Directory: dist

# 4. Add Environment Variable:
   VITE_API_URL = https://your-backend-url.onrender.com

# 5. Deploy
```

Backend Deployment (Render)

```
# 1. Go to render.com → New Web Service → Connect GitHub repo

# 2. Set these settings:
   Root Directory: KrishiMitra/server
   Build Command: npm install
   Start Command: node index.js

# 3. Add Environment Variables:
   DATABASE_URL = your_supabase_connection_string
   JWT_SECRET = your_secret
   OPENWEATHER_API_KEY = your_key
   ML_SERVICE_URL = https://your-ml-service.onrender.com
   AGMARKNET_API_KEY = your_key

# 4. Deploy
```

ML Service Deployment (Render with Docker)

```
# 1. Go to render.com → New Web Service → Connect GitHub repo

# 2. Set these settings:
   Root Directory: KrishiMitra/ml_service
   Environment: Docker

# The Dockerfile handles everything. Be aware:
# - TensorFlow makes the Docker image ~2GB
# - First cold start may take 30-60 seconds
# - Free tier services spin down after inactivity

# 3. Deploy
```

Important Production Changes

- Restrict CORS to only allow your frontend domain
- Use a strong, unique JWT_SECRET (32+ random characters)
- Set NODE_ENV=production for Express optimizations
- Consider connection pooling for the database
- Set up monitoring and logging (e.g., Sentry for error tracking)

KrishiMitra

AI-Powered Smart Farming Platform

Complete Project Documentation — End